Security Incident Report: Website Compromise via Brute Force

Malicious redirect injected into yummyrecipesforme.com

Type	Security incident report
Tools	tcpdump, sandbox environment
Protocols	HTTP, DNS
Skills	Traffic analysis, incident documentation, recognising brute-force attacks, remediation planning
Source	Google Cybersecurity Professional Certificate, Course 3 (*Connect and Protect: Networks and Network Security*)

Situation

A former employee used a **brute-force attack** to get into the administrator account of yummyrecipesforme.com. They tried known default passwords until one worked. They then added JavaScript to the site's source code that prompted visitors to download a fake "browser update" file, and changed the admin password to lock the owner out. Visitors who ran the file were redirected to **greatrecipesforme.com**, a fake site hosting malware, and their computers slowed down.

Section 1: Network protocol involved

The main protocol involved is HTTP (Hypertext Transfer Protocol), at the application layer.

When I loaded the site in a sandbox while running tcpdump, the capture showed the browser requesting the webpage over HTTP. The malicious executable was then **delivered to my computer over HTTP**. **DNS** was used too: the browser resolved yummyrecipesforme.com first and then greatrecipesforme.com. However, DNS itself was working correctly and wasn't the cause of the problem. It's also worth noting that the site used plain HTTP, not HTTPS, so the traffic wasn't encrypted.

Section 2: Incident documentation

Detection. A few hours after the attack, several customers emailed the helpdesk. They said the site had prompted them to download a file for "free recipes", and that afterwards the site's address changed and their computers became slow. When the website owner tried to log in to the admin panel, they found they were locked out, and contacted the hosting provider.

Investigation. In a **sandbox environment**, so the company network wasn't at risk, I ran **tcpdump**, opened yummyrecipesforme.com, accepted the download and ran the file. The browser was redirected to greatrecipesforme.com. The tcpdump log showed this sequence:

1. DNS query for yummyrecipesforme.com, and DNS reply with the correct IP.
2. HTTP request for the yummyrecipesforme.com webpage.
3. The malicious executable downloaded over HTTP.
4. A new DNS query for **greatrecipesforme.com**, and the DNS reply.
5. HTTP request to the IP address of greatrecipesforme.com. This is where the traffic was redirected.

Root cause. A senior analyst reviewed the website source code and found the injected JavaScript. Analysing the downloaded file showed a script that redirects browsers to greatrecipesforme.com. The team concluded that the attacker got admin access by **brute-forcing the admin account, which still used the default password**. There were **no controls to limit failed login attempts**.

Impact. The website's integrity was compromised, customers' computers were infected, the owner lost administrative control, and the business's reputation was damaged.

Section 3: Remediation for brute-force attacks

Main recommendation: strong authentication controls on the admin account.

- **Remove default credentials:** change every default password immediately and make this part of the deployment checklist.
- **Limit login attempts:** lock the account temporarily or add increasing delays after a few failed attempts. This makes brute-force attacks impractical.
- **Require multi-factor authentication** on the admin panel, so a guessed password alone isn't enough to log in.
- **Alert on admin logins**, including the source IP, device and location, so suspicious access is spotted quickly.

Note: forcing frequent password changes is *not* recommended. Current NIST guidance (SP 800-63B) advises changing passwords only when there's evidence of compromise, and putting the effort into password strength, breached-password screening and MFA.

Related actions for this incident:

- **Offboarding:** remove all access for departing employees on their last day. This attack came from a former insider.
- **Recovery:** restore the site from a known-good backup, regain control of the admin account through the hosting provider, and check that the source code is clean.
- **Customer communication:** warn affected users to remove the downloaded file and scan their devices.
- **Enable HTTPS** on the website.

Reflection

The capture showed the full attack chain in a few lines of DNS and HTTP traffic. The root cause, though, was an identity weakness: a default password and no lockout. Network evidence explained *what* happened; access controls are what would have *prevented* it.